

IDOR Response Platform - Final Technical Report

IDOR Response Platform - Final Technical Report

Generated at: 2026-06-10T03:04:25.601559+00:00

1. Challenge Context

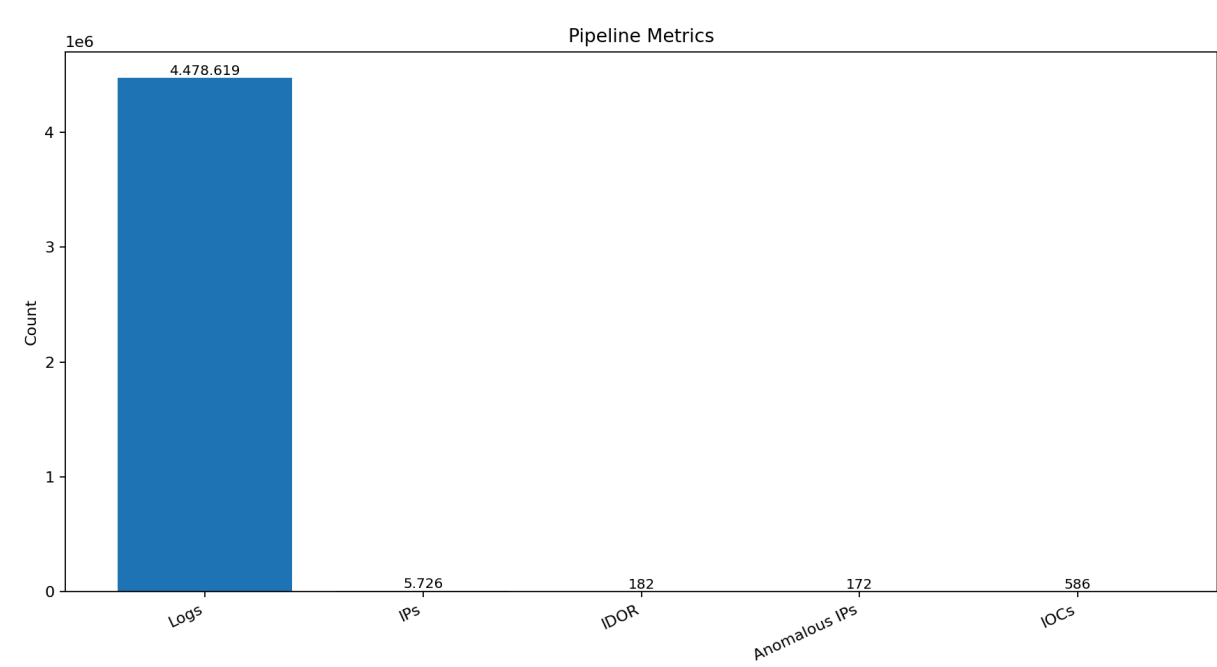
This report documents an end-to-end Digital Forensics and Incident Response platform for investigating IDOR - Insecure Direct Object Reference - exploitation patterns.

The challenge required a solution that could classify incidents, generate hypotheses, prioritize investigation, identify patient zero, explain automation, map MITRE ATT&CK concepts at a high level, recommend containment, support simulated human intervention, estimate response metrics and expose dashboards with evidence that the solution worked.

2. Dataset

Field	Value
---	---
Logs processed	4.478.619
IPs analyzed	5.726
IDOR findings	182
Anomalous IPs	172
IOCs generated	586
Attack start	2020-10-01 00:00:00.000000
Attack end	2020-12-31 23:58:00.000000
Invoices involved	10221
Attack events	96829
Tokens observed	35

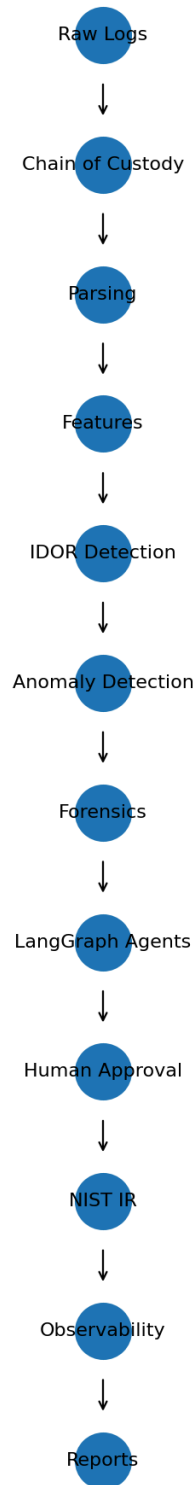
The dataset was processed from raw web access logs. URI parsing extracted invoice identifiers, site identifiers and token-like fields.



3. Architecture

The architecture follows a modular DFIR pipeline from evidence preservation to executive reporting.

IDOR Response Platform - Architecture



Key decisions include Polars for large-scale processing, Parquet for columnar persistence, Pydantic for typed contracts, Isolation Forest for anomaly detection, LangGraph for agent orchestration and Streamlit for analyst interaction.

4. Data Pipeline

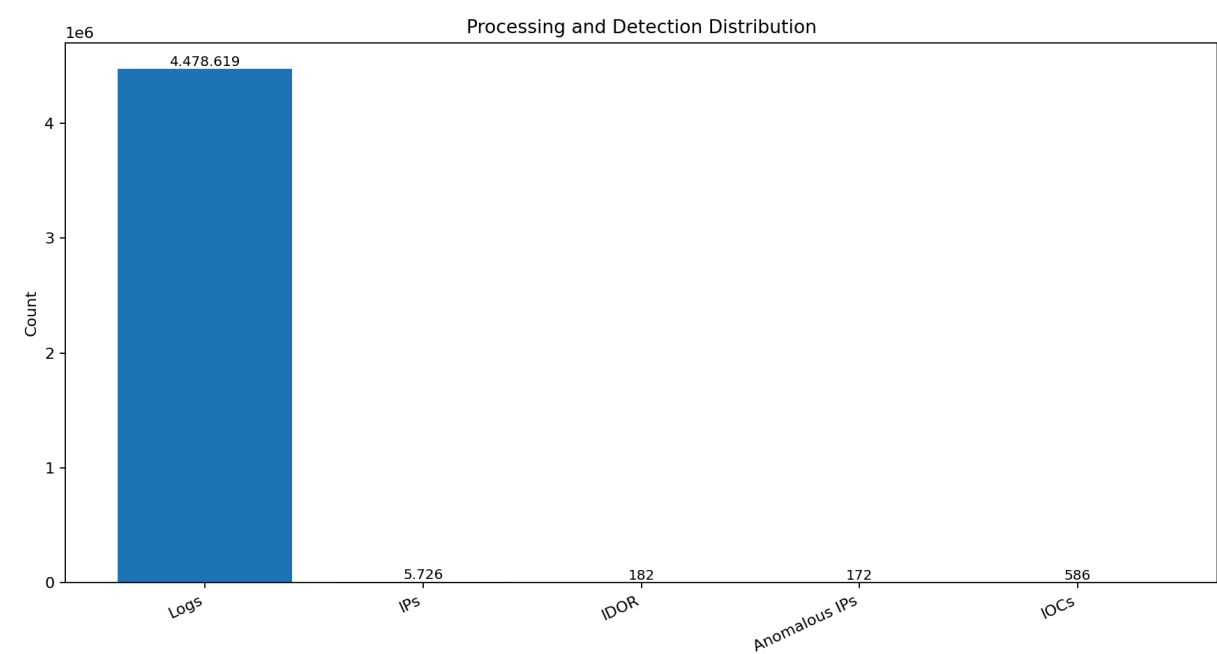
Sprint 1.1 implemented forensic ingestion and chain of custody. Sprint 1.2 implemented feature engineering and detection foundations. Sprint 1.3 generated forensic evidence, IOCs and attack timelines.

5. IDOR Detection

The deterministic detector identifies high-volume invoice access, high invoice diversity, direct object enumeration, token reuse and suspicious endpoint concentration.

6. Anomaly Detection

Isolation Forest is used as a complementary signal to prioritize unusual IP behavior. It is not the single source of truth; it supports convergence with deterministic IDOR features.



7. Forensic Investigation

| Field | Value |

| --- | --- |

| Patient zero candidate | 204.210.158.207 |

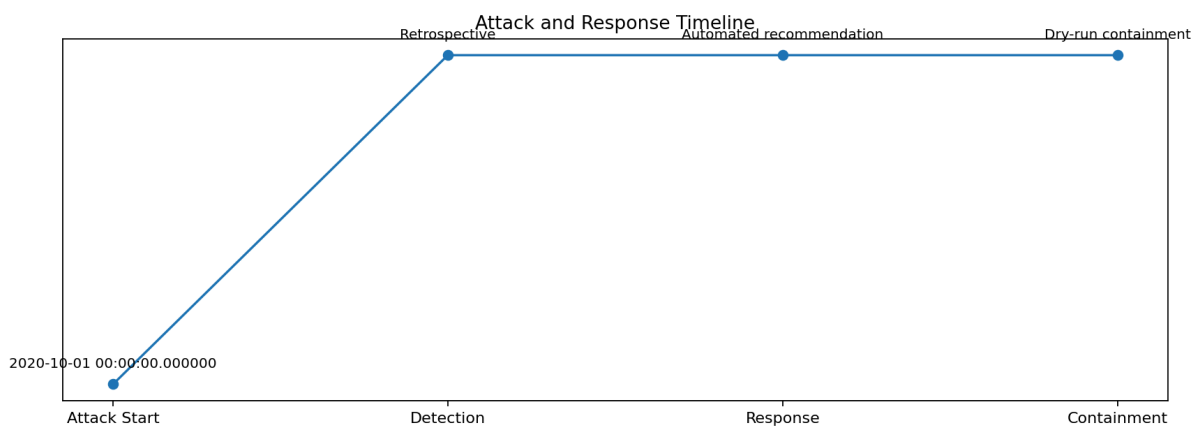
| Attack start | 2020-10-01 00:00:00.000000 |

| Attack end | 2020-12-31 23:58:00.000000 |

| Automated behavior | True |

| Invoices involved | 10221 |

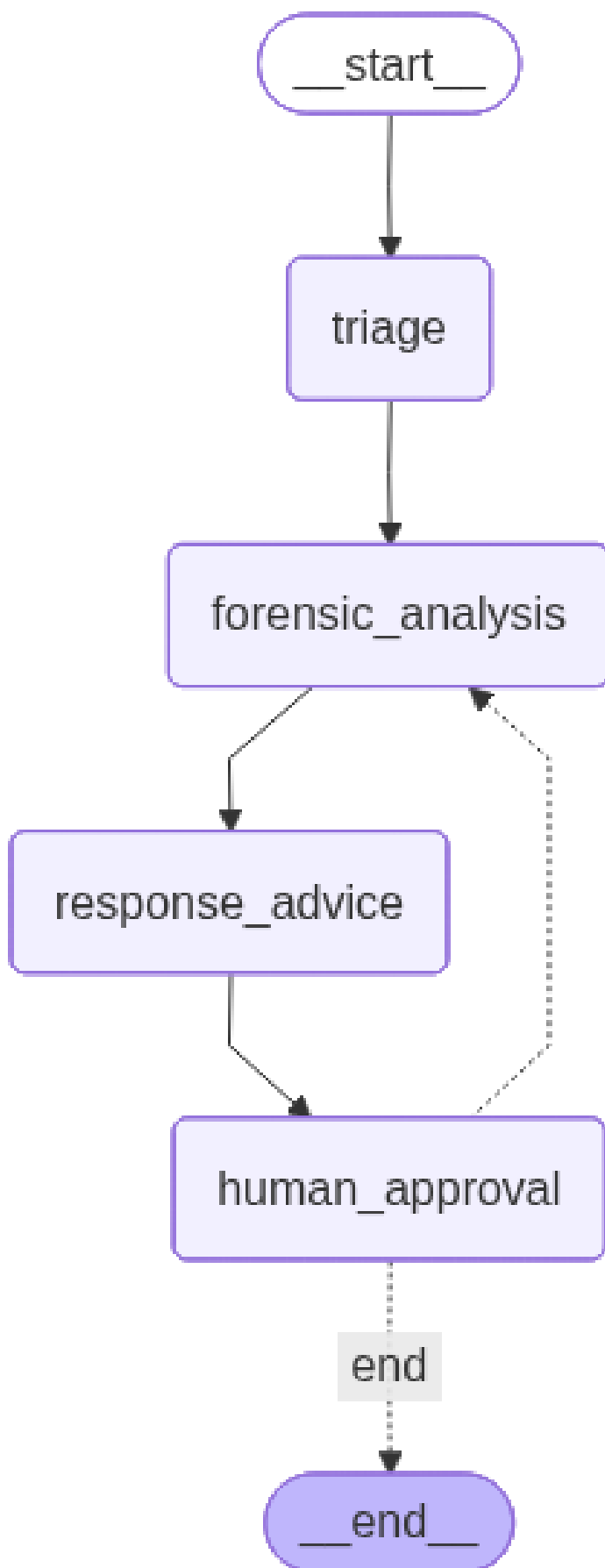
| Attack events | 96829 |



The forensic layer reconstructs the attack window, identifies IOCs, estimates impact and maps TTPs at a high level to MITRE ATT&CK.

8. Agentic Investigation

The platform includes Triage, Forensic Analyst, Response Advisor and Human Approval agents. Each agent maps to a specific challenge requirement.



9. Human-in-the-loop Validation

The platform validates simulated human intervention.

Scenario	Evidence	Result
----------	----------	--------

---	---	---
-----	-----	-----

approve	human_loop_count = 0, decision_log = 4	Linear approval
---------	--	-----------------

reject	workflow_stage = rejected, decision_log = 4	Containment interrupted
--------	---	-------------------------

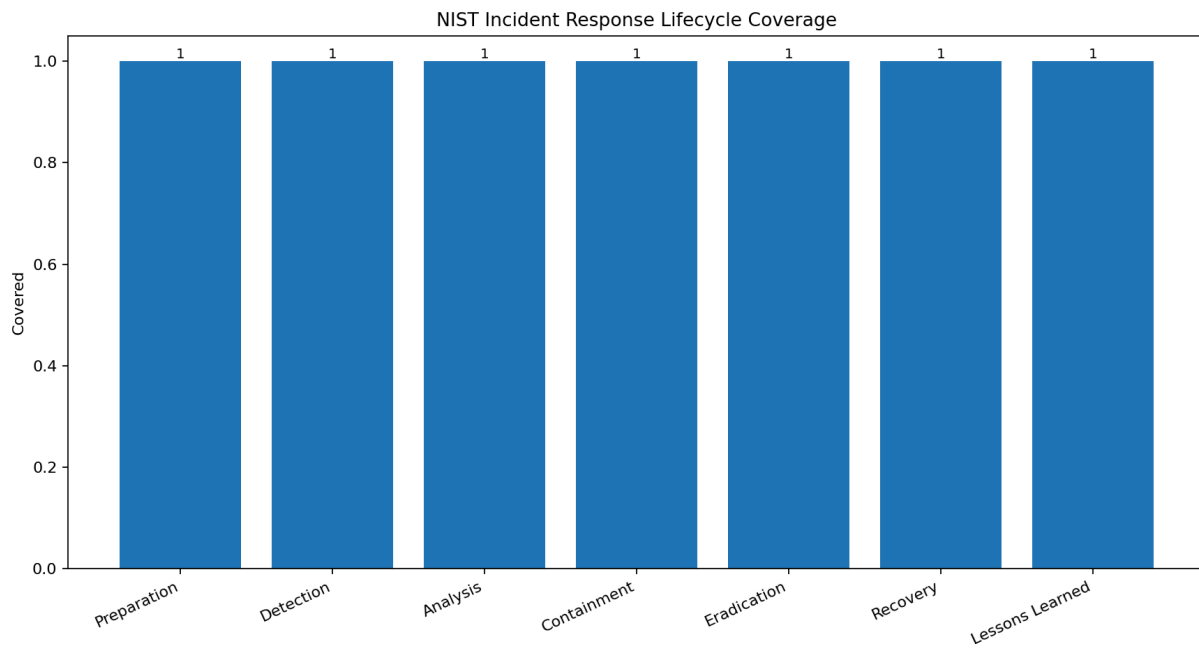
modify	modified_action_plan exists, decision_log = 4	Plan changed
--------	---	--------------

request_more_evidence	human_loop_count = 1, decision_log = 7	Workflow re-entry proven
-----------------------	--	--------------------------

The request_more_evidence scenario proves that the graph re-enters forensic analysis before approving containment.

10. NIST Incident Response

The NIST layer produces analysis, containment, eradication, response metrics and root cause analysis.



TTD is retrospective because the dataset is historical. TTR and TTC are zero in the automated dry-run cycle because response recommendation and simulated containment happen during the same run.

11. Observability

| Field | Value |

| --- | --- |

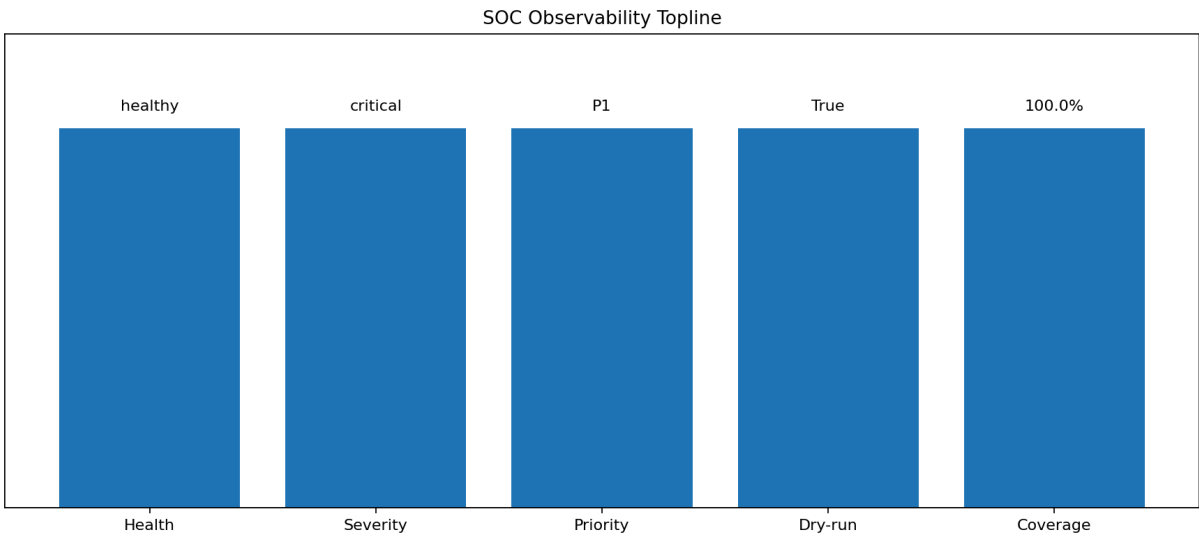
| Health | healthy |

| Severity | critical |

| Priority | P1 |

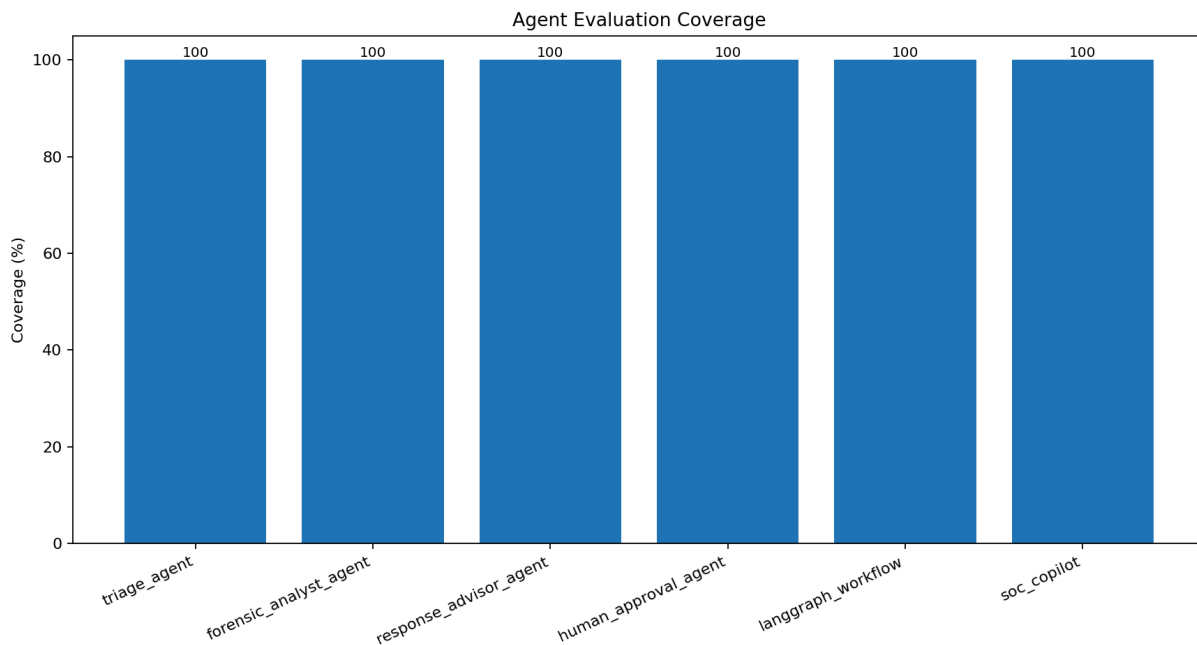
| Dry-run | True |

| Coverage | 100.0% |



12. Agent Evaluation

Field	Value
---	---
Overall coverage	100.0%
Passed	15
Partial	0
Failed	0



The validation suite proves that the platform covers classification, prioritization, forensic reasoning, patient zero identification, automation assessment, containment planning, human approval and workflow re-entry.

13. Limitations

The platform intentionally runs containment in dry-run mode. It does not execute destructive production actions. Production integrations such as WAF, SIEM, RBAC and cloud deployment are backlog items.

14. Future Work

Future work includes Docker Compose, CI/CD, Prometheus/Grafana, secrets management, RBAC, authentication, persistent vector database, external MCP servers, SIEM integration and WAF integration.

15. Final Delivery Package

Sprint 4.0 adds the investigation question bank, demo outputs, technical pitch deck, robust technical report, executive summary and evidence appendix.

Appendix Note 1

This section preserves design rationale and validation evidence for review. The core implementation is reproducible through pytest, `python -m src.app` and Streamlit validation.

Appendix Note 2

This section preserves design rationale and validation evidence for review. The core implementation is reproducible through pytest, `python -m src.app` and Streamlit validation.

Appendix Note 3

This section preserves design rationale and validation evidence for review. The core implementation is reproducible through pytest, `python -m src.app` and Streamlit validation.

Appendix Note 4

This section preserves design rationale and validation evidence for review. The core implementation is reproducible through pytest, `python -m src.app` and Streamlit validation.

Appendix Note 5

This section preserves design rationale and validation evidence for review. The core implementation is reproducible through pytest, `python -m src.app` and Streamlit validation.

Appendix Note 6

This section preserves design rationale and validation evidence for review. The core implementation is reproducible through pytest, `python -m src.app` and Streamlit validation.

Appendix Note 7

This section preserves design rationale and validation evidence for review. The core implementation is reproducible through pytest, `python -m src.app` and Streamlit validation.

Appendix Note 8

This section preserves design rationale and validation evidence for review. The core implementation is reproducible through pytest, `python -m src.app` and Streamlit validation.

Appendix Note 9

This section preserves design rationale and validation evidence for review. The core implementation is reproducible through pytest, `python -m src.app` and Streamlit validation.

Appendix Note 10

This section preserves design rationale and validation evidence for review. The core implementation is reproducible through pytest, `python -m src.app` and Streamlit validation.